# FEATURE ENGINEERING AND DATA PREPROCESSING

**NAME - Tejas Mali**

**ROLL NO. - 25**

**PRN - 202401040083**

**DIVISION - AIML**

## Carbon Footprint Prediction

## Problem Statement

To develop a machine learning model that can predict an individual's carbon footprint based on lifestyle activities such as transport, diet, electricity consumption, and waste generation.

# 1. Aim

To perform feature engineering and data preprocessing on a carbon-emission dataset and prepare a clean, numerical dataset suitable for training a machine learning regression model.

# 2. Dataset Overview

The dataset provided for this practical is the uploaded file “Carbon Emission.csv”. The working notebook previously reported 10,000 rows and 20 columns, with the target column named “CarbonEmission”. This differs from the brief's stated 2,001 rows and 6 columns; confirm the required dataset with the instructor if the assignment specification expects the smaller dataset.

- Target variable: CarbonEmission (numerical carbon-emission value).
- Feature examples used in the notebook: Diet, Transport, Energy Source, Vehicle Type, Vehicle Monthly Distance Km, Waste Generated Kg, Recycling, and Social Activity.
- Task type: Regression, because the target is a numerical value.

# 3. Software and Libraries Required

- Python 3
- Jupyter Notebook or Google Colab
- pandas and NumPy
- Matplotlib and Seaborn
- scikit-learn

If using Google Colab, upload “Carbon Emission.csv” using the Files sidebar or run:

```
from google.colab import files
uploaded = files.upload()
```

# 4. Section 1 — Dataset Creation and Display

Aim: Load the CSV file, inspect its dimensions, identify data types, and summarize its contents.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

df = pd.read_csv("Carbon Emission.csv")

print("First 5 rows:")
display(df.head())

print("Last 5 rows:")
display(df.tail())

print("Dataset shape:", df.shape)
print("Column names:", df.columns.tolist())
df.info()

print("Statistical summary:")
```

```
display(df.describe(include="all"))

numerical_features = df.select_dtypes(
    include=["int64", "float64"]
).columns.tolist()

categorical_features = df.select_dtypes(
    include=["object", "category"]
).columns.tolist()

print("Numerical features:", numerical_features)
print("Categorical features:", categorical_features)
print("Target variable: CarbonEmission")
```

Expected outcome: The dataset is loaded and its structure, data types, and feature groups are displayed.

## 5. Section 2 — Missing-Value Handling

Aim: Identify missing data and handle missing categorical values appropriately.

```
print("Missing values before handling:")
print(df.isnull().sum())

plt.figure(figsize=(10, 5))
sns.heatmap(df.isnull(), cbar=False, yticklabels=False)
plt.title("Missing Values Before Imputation")
plt.show()

if "Vehicle Type" in df.columns:
    df["Vehicle Type"] = df["Vehicle Type"].fillna("Not Applicable")

print("Missing values after handling:")
print(df.isnull().sum())
print("Total missing values:", df.isnull().sum().sum())
```

Note: The notebook reported missing values in Vehicle Type. The label “Not Applicable” should be used only when a missing vehicle type can reasonably mean that the category does not apply.

## 6. Section 3 — Numerical Data Visualization

Aim: Visualize the target distribution, potential outliers, and relationships between numerical features.

```
numerical_features = df.select_dtypes(
    include=["int64", "float64"]
).columns.tolist()

print("Numerical features:", numerical_features)

plt.figure(figsize=(8, 5))
sns.histplot(df["CarbonEmission"], kde=True)
plt.title("Distribution of Carbon Emission")
plt.xlabel("Carbon Emission")
plt.ylabel("Frequency")
plt.show()

plt.figure(figsize=(8, 5))
sns.boxplot(y=df["CarbonEmission"])
plt.title("Box Plot of Carbon Emission")
plt.show()
```

```
plt.figure(figsize=(10, 6))
sns.heatmap(df[numerical_features].corr(), annot=True,
            cmap="coolwarm", fmt=".2f")
plt.title("Correlation Heatmap")
plt.show()

if "Vehicle Monthly Distance Km" in df.columns:
    plt.figure(figsize=(8, 5))
    sns.scatterplot(data=df, x="Vehicle Monthly Distance Km",
                    y="CarbonEmission")
    plt.title("Vehicle Distance vs Carbon Emission")
    plt.show()

if "Waste Generated Kg" in df.columns:
    plt.figure(figsize=(8, 5))
    sns.scatterplot(data=df, x="Waste Generated Kg",
                    y="CarbonEmission")
    plt.title("Waste Generated vs Carbon Emission")
    plt.show()
```

Expected outcome: Histograms, box plots, scatter plots, and a correlation heatmap are generated.

## 7. Section 4 — Categorical Data Visualization

Aim: Explore the distribution of lifestyle categories and compare their average carbon emissions.

```
if "Transport" in df.columns:
    plt.figure(figsize=(8, 5))
    sns.countplot(data=df, x="Transport")
    plt.title("Distribution of Transport Modes")
    plt.xticks(rotation=30)
    plt.show()

if "Diet" in df.columns:
    plt.figure(figsize=(8, 5))
    sns.barplot(data=df, x="Diet", y="CarbonEmission")
    plt.title("Average Carbon Emission by Diet")
    plt.xticks(rotation=30)
    plt.show()

if "Energy Source" in df.columns:
    energy_counts = df["Energy Source"].value_counts()
    plt.figure(figsize=(7, 7))
    plt.pie(energy_counts, labels=energy_counts.index,
            autopct="%1.1f%%", startangle=90)
    plt.title("Distribution of Energy Sources")
    plt.show()

if "Recycling" in df.columns:
    plt.figure(figsize=(8, 5))
    sns.barplot(data=df, x="Recycling", y="CarbonEmission")
    plt.title("Average Carbon Emission by Recycling")
    plt.show()

if "Vehicle Type" in df.columns:
    plt.figure(figsize=(8, 5))
    sns.countplot(data=df, x="Vehicle Type")
    plt.title("Distribution of Vehicle Types")
    plt.xticks(rotation=30)
    plt.show()

if "Social Activity" in df.columns:
```

```
    plt.figure(figsize=(8, 5))
    sns.barplot(data=df, x="Social Activity", y="CarbonEmission")
    plt.title("Average Carbon Emission by Social Activity")
    plt.xticks(rotation=30)
    plt.show()
```

Interpretation: These plots show distributions and associations; they do not by themselves prove causation.

## 8. Section 5 — Categorical Encoding

Aim: Demonstrate label encoding, ordinal encoding, one-hot encoding, and a simple binary representation. This section creates a separate demonstration copy; the final dataset is prepared independently in Section 8.

```
from sklearn.preprocessing import LabelEncoder, OrdinalEncoder

df_encoded = df.copy()

# 1. Label encoding
for col in ["Transport", "Energy Source"]:
    if col in df_encoded.columns:
        le = LabelEncoder()
        df_encoded[col + "_encoded"] = le.fit_transform(
            df_encoded[col].astype(str)
        )

# 2. OrdinalEncoder demonstration for diet categories
if "Diet" in df_encoded.columns:
    print("Diet categories:", df_encoded["Diet"].unique())
    ordinal_encoder = OrdinalEncoder()
    df_encoded["Diet_ordinal"] = ordinal_encoder.fit_transform(
        df_encoded[["Diet"]].astype(str)
    )

# 3. One-hot encoding
one_hot_columns = [
    col for col in ["Vehicle Type", "Social Activity"]
    if col in df_encoded.columns
]
df_encoded = pd.get_dummies(
    df_encoded, columns=one_hot_columns, dtype=int
)

# 4. Simple binary representation of Recycling categories
if "Recycling" in df_encoded.columns:
    recycling_codes = pd.factorize(
        df_encoded["Recycling"].astype(str)
    )[0]
    binary_width = max(1, int(recycling_codes.max()).bit_length())
    for i in range(binary_width):
        df_encoded[f"Recycling_bit_{i}"] = (
            recycling_codes >> i
        ) & 1

display(df_encoded.head())
print("Encoded dataset shape:", df_encoded.shape)
```

Note: The automatically assigned diet numbers do not imply a scientifically justified ranking. For production modeling, fit encoders on training data only and reuse them for test/new data.

## 9. Section 6 — Feature Scaling

Aim: Compare four commonly used feature scaling methods on numerical input features.

```
from sklearn.preprocessing import (
    StandardScaler, MinMaxScaler, RobustScaler, Normalizer
)

numerical_features = df.select_dtypes(
    include=["int64", "float64"]
).columns.tolist()
numerical_features = [
    col for col in numerical_features if col != "CarbonEmission"
]

X = df[numerical_features].copy()
X = X.fillna(X.median())

X_standard = pd.DataFrame(
    StandardScaler().fit_transform(X), columns=numerical_features
)
X_minmax = pd.DataFrame(
    MinMaxScaler().fit_transform(X), columns=numerical_features
)
X_robust = pd.DataFrame(
    RobustScaler().fit_transform(X), columns=numerical_features
)
X_normalized = pd.DataFrame(
    Normalizer().fit_transform(X), columns=numerical_features
)

print("StandardScaler:")
display(X_standard.head())
print("MinMaxScaler:")
display(X_minmax.head())
print("RobustScaler:")
display(X_robust.head())
print("Normalizer:")
display(X_normalized.head())
```

StandardScaler centers each feature and scales it to unit variance; MinMaxScaler maps values to a chosen range; RobustScaler uses median and interquartile range; Normalizer scales each row to unit norm.

## 10. Section 7 — Feature Engineering

Aim: Create additional features, demonstrate a power transformation and PCA, and examine potential outliers.

```
from sklearn.preprocessing import PowerTransformer, StandardScaler
from sklearn.decomposition import PCA

df_fe = df.copy()

# Feature creation
if "Vehicle Monthly Distance Km" in df_fe.columns:
    df_fe["Vehicle_Distance_Per_Day"] = (
        df_fe["Vehicle Monthly Distance Km"] / 30
    )

if "Waste Generated Kg" in df_fe.columns:
    df_fe["Waste_Generated_Per_Day"] = (
        df_fe["Waste Generated Kg"] / 30
    )
```

```
# Yeo-Johnson transformation of the target for demonstration
transformer = PowerTransformer(method="yeo-johnson")
df_fe["CarbonEmission_Transformed"] = transformer.fit_transform(
    df_fe[["CarbonEmission"]]
)

# PCA on numerical predictors only
pca_features = df_fe.select_dtypes(
    include=["int64", "float64"]
).columns.tolist()
pca_features = [
    c for c in pca_features
    if c not in ["CarbonEmission", "CarbonEmission_Transformed"]
]

if len(pca_features) >= 2:
    X_pca = df_fe[pca_features].copy()
    X_pca = X_pca.fillna(X_pca.median())
    X_scaled_pca = StandardScaler().fit_transform(X_pca)
    pca = PCA(n_components=1)
    df_fe["Lifestyle_PCA"] = pca.fit_transform(X_scaled_pca)
    print("PCA explained variance:",
          round(pca.explained_variance_ratio_[0], 4))

# IQR outlier analysis on target (demonstration only)
Q1 = df_fe["CarbonEmission"].quantile(0.25)
Q3 = df_fe["CarbonEmission"].quantile(0.75)
IQR = Q3 - Q1
lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

outliers = df_fe[
    (df_fe["CarbonEmission"] < lower_bound) |
    (df_fe["CarbonEmission"] > upper_bound)
]
print("Potential target outliers:", len(outliers))

df_fe["CarbonEmission_Capped"] = df_fe["CarbonEmission"].clip(
    lower=lower_bound, upper=upper_bound
)

display(df_fe.head())
print("Feature-engineered dataset shape:", df_fe.shape)
```

Important: The target transformation and target capping are demonstrations, not automatic recommendations. For a predictive model, do not remove or cap genuine high-emission targets without justification. Fit transformations and PCA on training data only.

## 11. Section 8 — Final Preprocessed Dataset

Aim: Prepare a clean dataset with encoded input features and preserve CarbonEmission as the numerical target.

```
from sklearn.preprocessing import StandardScaler

final_df = df.copy()
target = "CarbonEmission"

# Fill categorical missing values
categorical_cols = final_df.select_dtypes(
```

```
    include=["object", "category"]
).columns.tolist()
for col in categorical_cols:
    final_df[col] = final_df[col].fillna("Unknown")

# Fill numerical predictor missing values
numerical_cols = final_df.select_dtypes(
    include=["int64", "float64"]
).columns.tolist()
for col in numerical_cols:
    if col != target:
        final_df[col] = final_df[col].fillna(
            final_df[col].median()
        )

# One-hot encode categorical predictors
final_df = pd.get_dummies(
    final_df, columns=categorical_cols, dtype=int
)

# Separate features and target
X = final_df.drop(columns=[target])
y = final_df[target]

# Scale numerical input columns
numeric_input_cols = X.select_dtypes(
    include=["int64", "float64"]
).columns.tolist()
if numeric_input_cols:
    scaler = StandardScaler()
    X[numeric_input_cols] = scaler.fit_transform(
        X[numeric_input_cols]
    )

final_preprocessed_df = X.copy()
final_preprocessed_df[target] = y

print("Final dataset shape:", final_preprocessed_df.shape)
print("Total missing values:",
      final_preprocessed_df.isnull().sum().sum())
display(final_preprocessed_df.head())

output_file = "final_carbon_footprint_dataset.csv"
final_preprocessed_df.to_csv(output_file, index=False)
print("Saved:", output_file)

# In Google Colab, download the file:
from google.colab import files
files.download(output_file)
```

Note: This is a practical demonstration. For model evaluation, split data into training and test sets first; fit imputation and scaling on the training set only. The target is kept unscaled here.

## 12. Conclusion

- Successfully loaded and explored the Carbon Emission dataset.
- Identified missing values and demonstrated an appropriate missing-value handling approach.
- Used numerical and categorical visualizations to inspect distributions and relationships in the data.
- Demonstrated label encoding, ordinal encoding, one-hot encoding, and binary representation for categorical variables.

- Compared StandardScaler, MinMaxScaler, RobustScaler, and Normalizer.
- Performed feature creation, Yeo–Johnson transformation, PCA, and IQR-based outlier analysis.
- Prepared and exported a preprocessed CSV file for further machine learning applications.
- The practical establishes a foundation for developing a regression model to estimate carbon emissions from lifestyle-related features.

## 13. Result

The carbon-emission dataset was explored and preprocessed using feature engineering techniques. The final preprocessed dataset was exported to CSV for use in subsequent regression-model development.

## 14. Important Notes Before Submission

- Run every notebook cell from top to bottom and confirm there are no errors.
- Verify that the dataset used matches the assignment specification. The uploaded file and stated dataset dimensions differ.
- Insert actual output screenshots/plots if your faculty requires them; this manual does not claim to include outputs that were not executed and verified.
- For real model evaluation, prevent data leakage by splitting into train/test sets before fitting preprocessing transformations.
- Do not use CarbonEmission or features derived from it as predictors when predicting CarbonEmission.

## 15. Google Collab link - https://colab.research.google.com/drive/1VJe-spL78CmYt3IvENg1XyrxeU0U55RA#scrollTo=vyeUSLUd40cK